Universidad De San Carlos De Guatemala
Facultad de ingeniería
Área Ingeniería
Laboratorio de Introducción a la Programación

Gestión de recursos y conversión de temperaturas

Nombre: José Pablo Pérez Monroy
Carné: 202405010
Aux: Diego Abraham Robles Meza
Sección de Laboratorio: A

Programación Orientada a Objetos: clase, objeto, atributo y método.

La Programación Orientada a Objetos (POO) es una forma de programar que se basa en cosas de la vida real. Una clase es como un molde o plantilla que nos sirve para crear algo. El objeto ya es la cosa creada en la memoria usando ese molde. Los atributos vienen siendo las características que tiene ese objeto (como el nombre o el código), y el método son las acciones o las funciones que ese objeto puede hacer.

Los cuatro pilares de la POO: encapsulamiento, herencia, polimorfismo y abstracción.

- Encapsulamiento: Es esconder los datos de una clase poniéndolos en private para que no los cambien desde afuera, usando solo los métodos get y set.

- Herencia: Es cuando una clase padre le hereda sus atributos y métodos a una clase hija para no repetir el mismo código.

- Polimorfismo: Es la capacidad que tiene un método o un objeto para comportarse de diferentes formas dependiendo de cómo se le llame.

- Abstracción: Es analizar algo de la vida real y solo sacar lo más importante o lo esencial para poder programarlo en código.

Diseño de clases para representar cursos y tareas académicas.

Para poder representar los cursos y tareas, es necesario abstraer estos conceptos y hacerles sus propias clases. Esto ayuda a que podamos crear muchos cursos y guardarlos en una lista, en vez de tener un montón de variables sueltas y desordenadas en el programa.

Java Swing: JFrame, JPanel, JLabel, JTextField, JButton, JTextArea y JOptionPane.
Java Swing es la librería que se usa para hacer interfaces gráficas en Java.

- JFrame: Es la ventana principal en sí.
- JPanel: Es como un contenedor adentro de la ventana para agrupar cosas.
- JLabel: Sirve para poner textos fijos que el usuario no puede borrar.
- JTextField: Es la cajita de texto donde el usuario puede escribir cosas.
- JButton: Son los botones que se pueden presionar.
- JTextArea: Es un cuadro de texto más grande para mostrar bastante información.
- JOptionPane: Son las ventanitas emergentes que saltan para dar mensajes o errores.

Manejo de eventos en Java: ActionListener y respuesta a botones

El ActionListener es lo que se usa para darle vida a los botones de la interfaz. Sirve para que el programa se quede "escuchando" o esperando a que el usuario le de clic a un botón, y cuando lo hace, ejecuta el código que pusimos adentro del método actionPerformed.

Validación de entradas numéricas y textuales en aplicaciones Java

Las validaciones sirven para revisar que el usuario no meta datos malos que puedan hacer que el programa falle o se cierre de la nada. Por ejemplo, se revisa que las cajitas no estén vacías con .isEmpty() o se usa un bloque try-catch por si el usuario escribe una letra donde iba un número para la temperatura, así atrapa el error y saca un aviso.

Explicación del diseño de clases utilizado

El diseño de la aplicación lo dividí en dos partes para tenerlo más ordenado usando programación orientada a objetos. Primero hice un paquete llamado "modelo", y ahí metí las clases curso y TareaAcademica. A estas clases les puse sus variables en privado y les generé su constructor y sus métodos getter y setter, para cumplir con el pilar del encapsulamiento.

Luego, en otro paquete llamado "vista", hice la clase VentanaPrincipal que hereda de la clase JFrame. En esta clase construí toda la parte visual instanciando los JLabel y JTextField a puro código usando el setLayout(null) y dándoles posiciones con setBounds, respetando la regla de no usar el GUI builder de arrastrar y soltar componentes. También ahí mismo metí la lógica de los ArrayList para ir guardando los objetos que se van registrando y la lógica de los botones para hacer las conversiones matemáticas de la temperatura.

Capturas del código fuente

Clase Curso (Modelo)

curso.java

```java
package modelo;

/**
 *
 * @author Joponcios
 */
public class curso {

    private int codigo;
    private String nombre , tutor;

    public curso(int codigo , String nombre , String tutor){
        this.codigo = codigo;
        this.nombre = nombre;
        this.tutor = tutor;
    }

    // getters y setters

    public int getCodigo(){
        return codigo;
    }

    public String getNombre(){
        return nombre;
    }

   public String getTutor(){
        return tutor;
    }

    // set

    public void setCodigo(int codigo){
        this.codigo = codigo;
    }

    public void setNombre(String nombre){
```

Clase TareaAcademica (Modelo)

curso.java × TareaAcademica.java ×

```java
package modelo;

/**
 *
 * @author Joponcios
 */
public class TareaAcademica {

    private String titulo , descripcion, fecha_entrega;

    public TareaAcademica(String titulo , String descripcion , String fecha_entrega){
        this.titulo = titulo;
        this.descripcion = descripcion;
        this.fecha_entrega = fecha_entrega;
    }

    // getters y setters

    public String getTitulo(){
        return titulo;
    }

    public String getDescripcion(){
        return descripcion;
    }

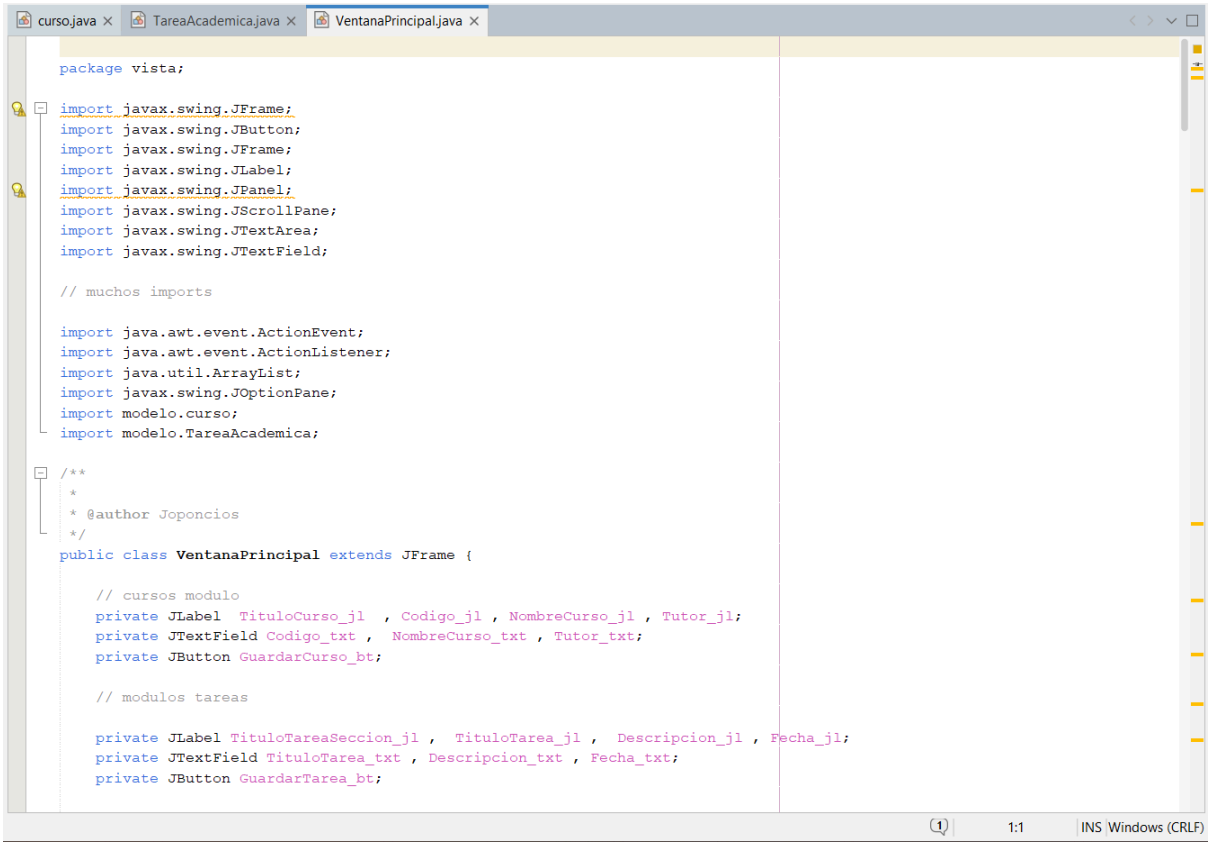
    public String getFecha_entrega(){
        return fecha_entrega;
    }

    // set

    public void setTitulo(String titulo){
        this.titulo = titulo;
    }

    public void setDescripcion(String descripcion){
```

Clase VentanaPrincipal (Vista)



curso.java × TareaAcademica.java × VentanaPrincipal.java ×

```java
    private JLabel TituloTemp_jl , Celcius_jl;
    private JTextField Celcius_txt;
    private JButton ConvertirF_bt , ConvertirK_bt;


    // muchos cosos

    private ArrayList<curso> Curso_lista = new ArrayList<>();
    private ArrayList<TareaAcademica> Tarea_lista = new ArrayList<>();

  public VentanaPrincipal() {
    setTitle("Gestor Académico y Conversor de Temperatura");
    setSize(700, 600);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLayout(null);

    Ventana();
    Botones();

  }
  private void Ventana(){

    // cursos modulo
    TituloCurso_jl = new JLabel("Registro del curso");
    TituloCurso_jl.setBounds(20, 10, 200, 20);
    add(TituloCurso_jl);

    Codigo_jl = new JLabel("Codigo :");
    Codigo_jl.setBounds(20, 40, 60, 20);
    add(Codigo_jl);

    Codigo_txt = new JTextField();
    Codigo_txt.setBounds(80, 40, 80, 20);
    add(Codigo_txt);

    NombreCurso_jl = new JLabel("Nombre: ");
    NombreCurso_jl.setBounds(180, 40, 60, 20);
    add(NombreCurso_jl);
```

1:1 INS Windows (CRLF)

curso.java × TareaAcademica.java × VentanaPrincipal.java ×

```java

  // poquitas tus lineas de codigo

  private void Botones(){
  // boton guardar curso
      GuardarCurso_bt.addActionListener(new ActionListener() {
          @Override
          public void actionPerformed(ActionEvent e) {
              // compruba si estan vacio los campos
              if (Codigo_txt.getText().isEmpty() || NombreCurso_txt.getText().isEmpty() || Tutor_txt.getText().isEmpty()) {
                  JOptionPane.showMessageDialog(null, "LLene los campos vacios", "Error", JOptionPane.ERROR_MESSAGE);
                  return;
              }

              try {
                  // verifica si es un numero
                  int codigo = Integer.parseInt(Codigo_txt.getText());
                  String nombre = NombreCurso_txt.getText();
                  String tutor = Tutor_txt.getText();

                  // crea objeto y lo guarda
                  curso Curso_nuevo = new curso(codigo, nombre, tutor);
                  Curso_lista.add(Curso_nuevo);
                  JOptionPane.showMessageDialog(null, "Curso guardado ");

                  // limpia las cajas de texto
                  Codigo_txt.setText("");
                  NombreCurso_txt.setText("");
                  Tutor_txt.setText("");
              } catch (NumberFormatException ex) {
                  JOptionPane.showMessageDialog(null, "el codigo debe ser numerico", "Error", JOptionPane.ERROR_MESSAGE);
              }
          }
      });

      // boton guardar tarera
      GuardarTarea_bt.addActionListener(new ActionListener() {
```

1:1 INS Windows (CRLF)

```java
// boton guardar tarera
GuardarTarea_bt.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        if (TituloTarea_txt.getText().isEmpty() || Descripcion_txt.getText().isEmpty() || Fecha_txt.getText().isEmpty
            JOptionPane.showMessageDialog(null, "llenar todos los campos", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        String titulo = TituloTarea_txt.getText();
        String descripcion = Descripcion_txt.getText();
        String fecha = Fecha_txt.getText();

        TareaAcademica Tarea_nueva = new TareaAcademica(titulo, descripcion, fecha);
        Tarea_lista.add(Tarea_nueva);
        JOptionPane.showMessageDialog(null, "tarea guardada");

        TituloTarea_txt.setText("");
        Descripcion_txt.setText("");
        Fecha_txt.setText("");
    }
});

//boton mostrar info
MostrarInfo_bt.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        Visual_area.setText(""); //limpiar antes de empezar

        Visual_area.append("cursos registrados \n");
        for (curso c : Curso_lista) {
            Visual_area.append("codigo: " + c.getCodigo() + " | nombre: " + c.getNombre() + " | tutor: " + c.getTutor

        }

        Visual_area.append("tareas registradas \n");
        for (TareaAacademica t : Tarea_lista) {
            Visual_area.append("titulo: " + t.getTitulo() + " | fecha: " + t.getFecha_entrega()+ "\n");
```

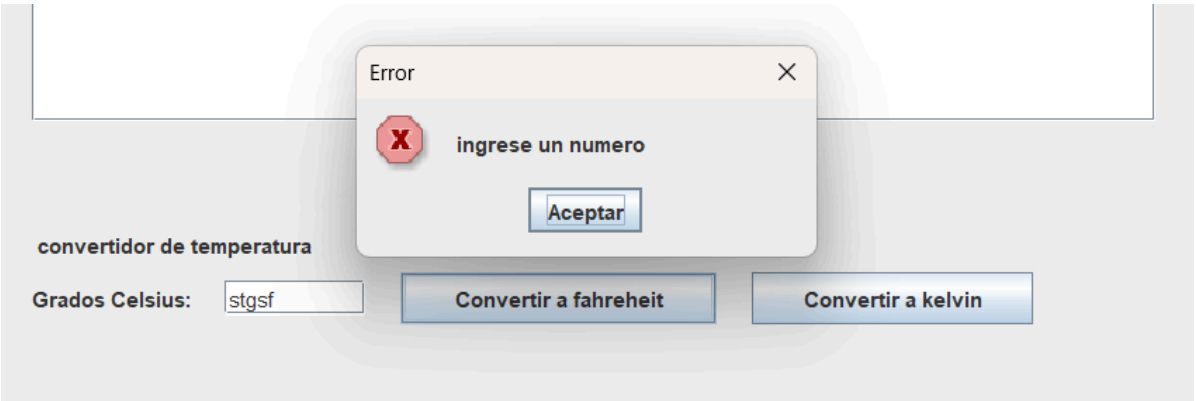
4. Programa en funcionamiento

Prueba de validación de campos vacíos:

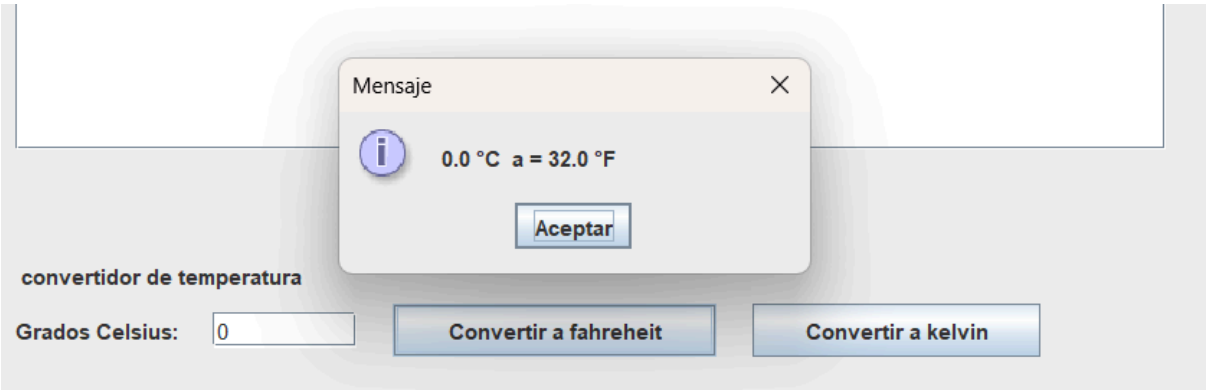


Prueba de error numérico:

Funcionamiento normal (Mostrar Información):



Módulo de Conversión de Temperaturas:



5. Conclusiones

- Hacer la interfaz gráfica escribiendo todo a puro código cuesta un poco más de trabajo y tiempo que solo usar el GUI builder, pero hacerlo así ayuda bastante a entender mejor cómo funciona realmente el acomodo de los componentes en Java y para qué sirven las coordenadas del setBounds.

- Usar la Programación Orientada a Objetos hace que el proyecto se vea mucho más limpio, ya que al separar los datos en las clases curso y TareaAcademica en un paquete distinto, la ventana principal solo se encarga de mostrar la información, lo que hace más fácil encontrar los errores.

- Las validaciones con los bloques try-catch y JOptionPane son de mucha importancia para manejar las excepciones, ya que evitan que el usuario "rompa" el programa si ingresa texto en lugar de números para hacer los cálculos de la temperatura.